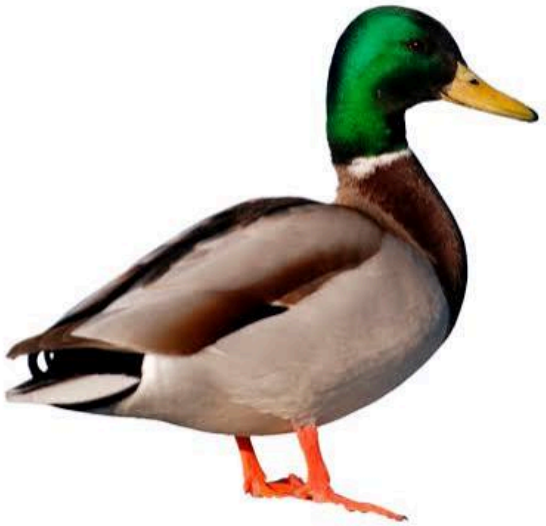
Ciência de Dados Aplicada ao Direito I

Aula 16

Funções



Prof. Bruno da Cunha de Oliveira - 2026/2

Feedback geral da fase 1 da APS

O feedback geral da fase 1 da APS está no Blackboard.

Ele reúne o que a banca e a turma comentaram sobre os pitches e os produtos, sem citar nenhum grupo, e termina com uma lista de conferência para o próximo pitch.

Quem apresentou o pitch da Acta

No meu registro, quem apresentou pela Acta foi a Maria Luiza. O grupo tem duas.

Maria Luiza J.	**Maria Luiza O. de M.**

Qual das duas?

No PrairieLearn: exercício 4

Insper

A mesma conta em vários lugares

Insper

A mesma conta, escrita uma vez só

Animação no navegador: `animacao-uma-vez.html`, cenário `fusao`.

Do pseudocódigo ao Python

```
FUNÇÃO prazo_em_dobro(dias):
    prazo = dias * 2
    DEVOLVER prazo

prazo_apelacao = prazo_em_dobro(15)
```

Para ver uma função por dentro, a mais simples: a que só dobra o prazo. Ela está em pseudocódigo. Escrevam a mesma função em Python.

def e return

RESPOSTA

```
def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo

prazo_apelacao = prazo_em_dobro(15)
```

`def` abre a definição, e a linha termina em dois pontos. O corpo fica indentado. `dias` é a entrada da função. `return` é o DEVOLVER: entrega a saída a quem chamou.

Rodar só a definição

```
def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo
```

Rodei só esta célula. O que aparece na tela?

Definir não executa

RESPOSTA

Nada.

A célula ensinou ao Python o que é `prazo_em_dobro`. A conta só roda quando alguém chama a função, com os parênteses: `prazo_em_dobro(15)`.

Uma chamada, passo a passo

Animação no navegador: `animacao-chamada.html`, cenário um.

Duas chamadas

```
def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo

prazo_apelacao = prazo_em_dobro(15)
prazo_embargos = prazo_em_dobro(5)
print(prazo_apelacao, prazo_embargos)
```

O que aparece na tela?

Duas chamadas, passo a passo

Animação no navegador: `animacao-chamada.html`, cenário `duas`.

Cada chamada tem os seus dias

RESPOSTA

```
30 10
```

Na primeira chamada, `dias` vale 15. Na segunda, vale 5. A definição é a mesma, e cada chamada devolve a sua resposta.

Parâmetro e argumento

```python
def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo

prazo_apelacao = prazo_em_dobro(15)
```

Elemento	Neste caso
parâmetro	`dias`, o nome escrito na definição
argumento	15, o valor que entra na chamada

A cada chamada, o parâmetro recebe o argumento.

O prazo do agravo de instrumento

```
def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo
```

O prazo do agravo de instrumento é de 15 dias. Escrevam a linha que calcula esse prazo em dobro e guarda o resultado em `prazo_agravo`.

Quanto vale `prazo_agravo` depois dessa linha?

Uma chamada para cada prazo

RESPOSTA

```
def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo

prazo_agravo = prazo_em_dobro(15)
```

`prazo_agravo` guarda 30. A regra do dobro está escrita uma vez, e cada prazo novo é uma chamada.

return e print

Uma função que só imprime

```python
def prazo_em_dobro(dias):
    prazo = dias * 2
    print(prazo)

total = prazo_em_dobro(15)
print(total - 12)
```

Troquei o `return` por `print`. Já se passaram 12 dias, e quero saber quantos restam. O que aparece na tela?

Só com print, passo a passo

Animação no navegador: `animacao-retornar-imprimir.html`, cenário `imprime`.

print mostra, return devolve

RESPOSTA

```
30
TypeError: unsupported operand type(s) for -:
'NoneType' and 'int'
```

O `print` de dentro mostrou 30 na tela, e a função não devolveu nada. Sem `return`, a chamada entrega `None`, e `None - 12` não é uma conta. Com `return prazo`, `total` guarda 30 e o programa mostra 18.

Com return, passo a passo

Animação no navegador: `animacao-retornar-imprimir.html`, cenário `retorna`.

Dentro e fora da função

Usar prazo fora da função

```
def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo

total = prazo_em_dobro(15)
print(prazo)
```

O que aparece na tela?

A variável de dentro, passo a passo

Animação no navegador: `animacao-escopo.html`, cenário `fora`.

A variável de dentro some

RESPOSTA

```
NameError: name 'prazo' is not defined
```

`prazo` nasceu dentro da função e deixou de existir quando ela terminou. Fora, existe só o que foi devolvido e guardado: `total`.

O mesmo nome, fora e dentro

```python
prazo = 5

def prazo_em_dobro(dias):
    prazo = dias * 2
    return prazo

total = prazo_em_dobro(15)
print(prazo, total)
```

O que aparece na tela?

O mesmo nome, passo a passo

Animação no navegador: `animacao-escopo.html`, cenário `mesmo-nome`.

Duas variáveis com o mesmo nome

RESPOSTA

```
5 30
```

O `prazo` de fora continua valendo 5. O de dentro é outra variável, que só existe durante a chamada.

Mais de uma entrada

Quantos dias restam do prazo

Para dizer quantos dias restam, a função recebe o prazo e os dias decorridos.

```
def dias_restantes(prazo, decorridos):
    restam = prazo - decorridos
    return restam

print(dias_restantes(30, 12))
```

```
18
```

Os parâmetros se separam por vírgula na definição, e os argumentos, por vírgula na chamada.

Os mesmos números em outra ordem

```
def dias_restantes(prazo, decorridos):
    restam = prazo - decorridos
    return restam

print(dias_restantes(30, 12))
print(dias_restantes(12, 30))
```

As duas chamadas usam os mesmos números. Mostram o mesmo valor?

A ordem dos argumentos

RESPOSTA

```
18
-18
```

O primeiro argumento vai para o primeiro parâmetro, e o segundo, para o segundo. Na segunda chamada, o prazo virou 12 e os dias decorridos viraram 30. O Python não avisa: o programa roda e responde errado.

Uma chamada com um argumento só

```
def dias_restantes(prazo, decorridos):
    restam = prazo - decorridos
    return restam

print(dias_restantes(30))
```

O que aparece na tela?

Cada parâmetro espera o seu argumento

RESPOSTA

```
TypeError: dias_restantes() missing 1 required
positional argument: 'decorridos'
```

A definição pede duas entradas, e a chamada trouxe uma. A mensagem diz qual faltou: `decorridos`.

Uma função que decide

Advogado no Juizado Especial Cível

Lei dos Juizados Especiais (Lei 9.099/1995). Art. 9º, caput.

> **Art. 9º** Nas causas de valor até vinte salários mínimos, as partes comparecerão pessoalmente, podendo ser assistidas por advogado; nas de valor superior, a assistência é obrigatória.

São duas respostas possíveis, e um teste separa uma da outra: o valor da causa, em salários mínimos.

Entradas e retorno da função do Juizado

Vou escrever a função `assistencia`, que aplica o art. 9º a uma causa. Antes do corpo, é preciso decidir o que ela recebe e o que ela devolve.

(a) Definam as entradas da função. Para cada uma, o nome, o tipo e o fato que representa.

(b) Definam o retorno da função: o tipo e a informação que ele transmite.

Uma entrada, um retorno

RESPOSTA

(a) `valor_sm`: `int`, o valor da causa em salários mínimos. É o único fato de que o art. 9º depende.

(b) O retorno é do tipo `str`: `"facultativa"` quando as partes podem comparecer sem advogado, e `"obrigatória"` quando a assistência é exigida.

A assistência em duas respostas

```
def assistencia(valor_sm):
    if valor_sm <= 20:
        resposta = "facultativa"
    else:
        resposta = "obrigatória"
    return resposta

print(assistencia(20), assistencia(21))
```

O que aparece na tela?

O valor da fronteira

RESPOSTA

```
facultativa obrigatória
```

20 passa no teste `valor_sm <= 20`, porque o artigo diz "até vinte". 21 não passa e cai no `else`. O `return` fica fora do `if`: qualquer que seja o caminho, a função devolve `resposta`.

O teto do Juizado Especial Cível

Lei dos Juizados Especiais (Lei 9.099/1995). Art. 3º, caput e inciso I.

> **Art. 3º** O Juizado Especial Cível tem competência para conciliação, processo e julgamento das causas cíveis de menor complexidade, assim consideradas:
>
> I - as causas cujo valor não exceda a quarenta vezes o salário mínimo; (...)

Pelo valor, uma causa de 50 salários mínimos passa do teto. A função precisa de uma terceira resposta.

Uma terceira resposta

Acrescentei um `if` para o teto. O que aparece na tela?

```
def assistencia(valor_sm):
    if valor_sm > 40:
        resposta = "acima do teto"
    if valor_sm <= 20:
        resposta = "facultativa"
    else:
        resposta = "obrigatória"
    return resposta
print(assistencia(50))
```

Duas perguntas independentes

RESPOSTA

```
obrigatória
```

50 passa em `valor_sm > 40`, e `resposta` vira `"acima do teto"`. Depois não passa em `valor_sm <= 20`, cai no `else`, e `resposta` vira `"obrigatória"`. O segundo `if` não sabe que o primeiro já respondeu.

elif: outra pergunta, só se a anterior deu False

```
def assistencia(valor_sm):
    if valor_sm <= 20:
        resposta = "facultativa"
    elif valor_sm <= 40:
        resposta = "obrigatória"
    else:
        resposta = "acima do teto"
    return resposta
```

`elif` é o SENÃO SE. O Python testa de cima para baixo e roda só o primeiro bloco cuja condição der `True`.

A ordem dos testes

Troquei a ordem dos dois primeiros testes. O que aparece na tela?

```
def assistencia(valor_sm):
    if valor_sm <= 40:
        resposta = "obrigatória"
    elif valor_sm <= 20:
        resposta = "facultativa"
    else:
        resposta = "acima do teto"
    return resposta
print(assistencia(15))
```

O primeiro True vence

RESPOSTA

```
obrigatória
```

15 cabe nos dois limites, e o Python fica com o primeiro teste que dá `True`. Para separar as faixas, o teste mais apertado vem antes: `<= 20`, e só depois `<= 40`.

O art. 171 como função

Insper

Quem preenche o campo configura?

```
caso = {"investigado": "Marina Alves", "configura": True}
```

Na triagem dos casos de estelionato, o campo `configura` chegava preenchido. Alguém leu os fatos do caso e aplicou o art. 171.

O que essa pessoa recebeu como entrada? O que ela devolveu?

Entradas e saída do art. 171

RESPOSTA

```
SE   obteve_vantagem_ilicita
 E   houve_prejuizo_alheio
 E   (induziu_em_erro OU manteve_em_erro)
 E   (usou_artificio OU usou_ardil
      OU usou_outro_meio_fraudulento)
```

As entradas são os fatos do caso, cada um VERDADEIRO ou FALSO. A saída é uma resposta só: configura ou não configura. Entradas e uma saída: é uma função.

Escrever a função do art. 171

Escrevam em Python a função `configura_estelionato`.

Ela recebe quatro entradas, cada uma `True` ou `False`: `vantagem` (houve vantagem ilícita), `prejuizo` (houve prejuízo alheio), `erro` (alguém foi induzido ou mantido em erro) e `fraude` (houve artifício, ardil ou outro meio fraudulento).

Ela devolve `True` se o caso configura estelionato, e `False` se não configura.

A condição é o que a função devolve

RESPOSTA

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude
```

`vantagem and prejuizo and erro and fraude` já vale `True` ou `False`, e o `return` entrega esse valor. Quem escreveu `if` e `else`, com `return True` num ramo e `return False` no outro, chegou à mesma resposta.

O caso de Marina e o caso de Otávio

Marina obteve vantagem ilícita, causou prejuízo e manteve a vítima em erro; não se apurou meio fraudulento. No caso de Otávio, os quatro fatos foram apurados. O que aparece na tela?

```
def configura_estelionato(vantagem, prejuizo, erro, fraude):
    return vantagem and prejuizo and erro and fraude

marina = configura_estelionato(True, True, True, False)
otavio = configura_estelionato(True, True, True, True)
print(marina, otavio)
```

A regra aplicada a dois casos

RESPOSTA

```
False True
```

Saber o caput de cor não diz se o caso de Marina configura estelionato. A função recebe os fatos e responde, e responde do mesmo jeito para o caso seguinte.